A Primer on KERI, ACDCs, and CESR

Daniel Hardman

2025-12-19

Abstract

An introduction to KERI, ACDCs, and CESR, explaining their roles in decentralized identity and verifiable data.

1. Administrative trust failure

The internet was not built with an identity layer. It was built as a network of machines, addressing endpoints rather than the human or legal entities controlling them. To fill this void, digital architects constructed the public key infrastructure (PKI), a system dependent on centralized certificate authorities (CAs) to attest to the binding between a cryptographic key and an entity. While functional for the early web, this administrative model of trust has proven brittle, expensive, and hard to govern in the face of modern scale and sophisticated adversaries.

The fundamental flaw in this architecture is the separation of the identifier from the cryptographic keys that control it. In the X.509 model, an identifier (like a domain name or email address) is a lease entry in a database. The cryptographic key is a separate entity. The binding between them is merely an assertion—a digital certificate—signed by a third party. This creates a root of trust that is external to the identity itself. We trust that a key controls an identifier not because of the math inherent in the identifier, but because we trust the administrator (the CA) who signed it.

1.1 Fragility of centralized roots

This reliance on administrative roots creates systemic fragility. If a CA is compromised, coerced, or negligent, it can issue valid certificates that assert bindings that are not true. This risk has actually materialized: incidents like the DigiNotar compromise demonstrated how a single breached CA can undermine trust for millions of users, allowing attackers to impersonate high-value domains like google.com [1]. Furthermore, the eventual distrust of Symantec's PKI due to negligent issuance [2], and TrustCor Systems due to opaque ties with intelligence services [3], proves that the risk is not just external attackers, but the administrators themselves. In this model, the security of the leaf (the user) is entirely dependent on the security of the root (the administrator).

To make matters more complicated, even if no mistakes occur, not every party has the same opinion as to the trustworthiness of a given administrator or the CAs that embody its policies — and opinions can change over time. This leads to situations where a CA-based ecosystem has different trust profiles in different jurisdictions [4]. For example, assertions rooted in the SHAKEN ecosystem mandated by US regulators are not accepted in Europe or Asia [5, 6]. The same phenomenon would plague the web, if it were not for a little-known group called the CA

Browser Forum, which acts as a centralizing body to align the world's browser manufactures around which certificate issuers are worthy of a lock icon in the browser's URL bar. This group is a new centralization, and it has its own challenges and failures [7, 8, 9].

The X.509 standard also lacks a native, low-latency, enforceable mechanism for revocation. When a private key is compromised, the certificate must be revoked. However, the mechanisms for this — Certificate Revocation Lists (CRLs) and the Online Certificate Status Protocol (OCSP) — are bolt-ons with important limitations. CRLs are heavy, bandwidth-intensive lists that scale poorly as revocation events grow [10]. OCSP requires the client to query a central responder for every validation, creating a privacy leak (the CA knows every site you visit) and a reliability bottleneck [11]. If the OCSP responder is offline, browsers often "fail open", meaning they accept the unverifiable certificate as valid because blocking traffic is considered too disruptive [12]. Stapling helps, but isn't always an option, and introduces additional complications.

To its credit, the PKI community has implemented certificate transparency (CT) to detect rogue issuance by logging certificates in append-only ledgers [13]. However, CT is a detection mechanism, not a prevention mechanism; fraudulent certificates may be usable for a time. And ironically, the implementation of CT has introduced a new form of centralization. Because browsers control the policy about which CT logs are trusted, the ecosystem has become heavily dependent on infrastructure provided by a few tech giants, effectively replacing one set of suboptimal gatekeepers with another [14, 15, 16, 17].

1.2 The persistence problem

More importantly, CT does not solve the underlying architectural issue of identity continuity. The administrative model of web identity is ephemeral. A certificate has an expiration date. When it expires, or when a key must be rotated due to compromise, the evidence about identity effectively resets. The new certificate contains a new public key and a new serial number. It is mathematically unrelated to the old certificate. The only link between them is the administrative procedure of the CA, which verifies the applicant again and issues a new assertion.

Specifically, the CA usually runs the ACME protocol [18] to reauthenticate the party asking for an updated certificate. While ACME provides continuity via a persistent account key, this continuity exists only in the eyes of the CA. The public just sees the issued certificate, which typically contains a rotated public key and no cryptographic link to predecessors. Consequently, the CA's reasons for asserting continuity remain opaque to the public, returning us to the administrative trust problem [19].

If an entity cannot prove *on its own* that it is the same entity that acted in a different context, its reputation remains perpetually derivative of external authorities that must reaffirm its existence and continuity [20]. This gives attackers a steady stream of indirect vulnerabilities to attack, and leads to a never-ending churn of evidence (a consequence of the X.509 standard's mandatory `Validity` fields (NotBefore / NotAfter) [21]). The result is elevated risk, unpredictable acceptance, and high maintenance costs for the whole ecosystem.

1.3 Requirements for a new stack

To solve these problems, we require an architecture that shifts the root of trust from the administrator to the cryptographic controller. We need a system where: 1. *The identifier is the root of*

trust: The binding between the ID and the key must be mathematical (and therefore, objectively provable and permanent), not administrative (and therefore, subjective, fragile, centralized, and temporary). 2. *History is auditable*: The system must provide a mechanism to detect conflicting histories (duplicity) immediately. Trust must be derived from the ability to verify the log of events, not the reputation of the log's host. 3. *Identity survives key rotation:* The system must allow keys to change without resetting the identity's history and without having to trust a party who claims it's safe to ignore a gap. 4. *Revocation is immediate and safe:* The status of keys and credentials must be verifiable without privacy leaks or "fail open" risks. 5. *Recovery is robust:* The system must allow straightforward, self-service, provable recovery from key loss without central intervention.

The technologies analyzed in this report—Key Event Receipt Infrastructure (KERI), Authentic Chained Data Containers (ACDC), and Composable Event Streaming Representation (CESR)—form a stack that meets these requirements. They replace administrative trust with cryptographic trust, enabling a Decentralized Key Management Infrastructure (DKMI) where the history of the identifier is the only authority required to verify it.

2. KERI

Key Event Receipt Infrastructure (KERI) is a protocol for decentralized key management. It is sometimes associated with blockchain technology because it involves cryptographic records and immutable histories. However, KERI is distinct. Traditional blockchains rely on a global consensus model, where every node must agree on the total ordering of all transactions in the network. The network thus faces a scalability bottleneck; the network can only process as many transactions as its consensus algorithm allows [22, 23].

KERI rejects the need for global consensus regarding identity. Instead, it uses microledgers called *key event logs* (KELs). Every identifier has its own independent KEL, with storage distributed and security enforced one identifier at a time. The ordering of events matters only relative to that specific identifier. Event 5 for Identifier A must come after Event 4 for Identifier A, but it has no required ordering relative to Event 5 for Identifier B. KERI naturally scales horizontally because there is no central choke point [24]. The lack of centralizing consensus and the lack of centralized storage also allow KERI to operate across jurisdictional boundaries with ease, and they change economics. Infrastructure gets cheaper because there's no massive system to maintain. Hacking incentives decay because any breach is likely to be limited in scope to a single identifier.

2.1 Autonomic identifiers

The core primitive of KERI is the *autonomic identifier* (AID). Unlike a domain name, which is rent-seeking text, or a UUID, which is arbitrary entropy, an AID is a *self-certifying identifier* (SCID). The identifier derives cryptographically from the initial public key (or set of keys) that controls it.

To create an AID, the controller generates key material. They then choose a derivation code (defined in CESR) that specifies the cryptographic algorithm (e.g., Ed25519, ECDSA secp256k1) used to transform the public portion of the key material into the identifier string.

KERI distinguishes between two modes of derivation: basic and transferable.

Basic derivation: In this mode, the identifier is just a digest (hash) of a single public key. This is similar to how many cryptocurrency addresses work. While simple, it is brittle. If the private key

is compromised or lost, the identifier must be abandoned. You cannot rotate the key because the identifier *is* the key digest; changing the key changes the identifier, which means you must abandon all reputation, credentials, and relationships associated with the original identifier. Basic AIDs are suitable only for ephemeral, short-lived use cases.

Transferable derivation: This is KERI's primary innovation for persistent identity. In this mode, the identifier is derived not just from one or more public keys, but from the entire *inception event*. The inception event contains the initial public key material *and* a cryptographic commitment to the *next* key material (pre-rotation). The identifier string remains constant even as the keys change, because the root of trust is the inception event, which establishes the rules for how the keys are allowed to evolve. Identity survives key rotation [25].

2.2 The key event log

The *key event log* (KEL) is the authoritative source of truth for an AID. It is an append-only chain of data structures (events) signed by the controller. When a verifier wants to know "Who controls this AID?" or "Is this signature valid?", they do not query a directory; they ingest the KEL.

The KEL is verified by replaying history to see if the currently claimed state derives correctly. The verifier starts at the inception event, verifies the signature, and then moves to the next event. Each subsequent event is cryptographically linked to the previous one via a hash of the predecessor, in an unbreakable chain of custody. If a single bit of a past event is altered, the hash links break, and the log is rejected as invalid.

The KEL supports three primary event types: 1. *Inception* (`icp`)*:* Birth of the identifier. 2. *Rotation* (`rot`)*:* Update of keys or configuration. 3. *Interaction* (`ixn`)*:* Anchoring of data (seals) without changing keys.

2.3 Pre-rotation

The hard problem of PKI is secure key rotation. If an attacker compromises your active private key, they can sign a transaction to rotate the key to one they control, effectively locking you out of your own identity. In standard PKI, there is no defense against this; a signature from the compromised key looks exactly like a signature from the legitimate owner.

KERI solves this through *pre-rotation*.

When a controller creates an event (say, the inception event), they must decide *now* what key they will use for the *next* rotation, and they must manage the next key differently from the current one (e.g., storing the next key offline or on a different device). Let's call the current key K1 and the next key K2. The controller includes the public key for K1 in the event so they can sign it. However, they do not include the public key for K2. Instead, they include a cryptographic hash (digest) of K2.

The security comes from the asymmetry in knowledge: the controller knows or has access to K2, while the world only sees its hash. An attacker who compromises K1 gains the ability to sign with that key, but cannot produce even the public key portion of K2; all they know is that if they had that key, it would hash to the committed value.

Pre-rotation establishes a firewall between day-to-day use and occasional governance. A controller can keep K1 on a production server while generating K2, hashing it, and immediately

storing it in an air-gapped, cold wallet. If the server is breached, the attacker steals K1 but cannot rotate to assume control of the identity. The legitimate owner can retrieve K2, rotate the keys, and regain control [26].

When the time comes to rotate, the controller creates a *rotation event*. In this event, they: 1. Reveal the public key for K2, thus proving that they know the governing secret that was committed to at a previous time via its hash. 2. Sign the event with the private key for K2. 3. Commit to a *new* future key (K3) by including its hash.

The verifier checks this logic: "Does the hash of this newly revealed key K2 match the commitment `n` that was recorded in the previous event?"

2.4 Weighted multisig and transparent, granular governance

Many systems support basic multisig (e.g., "requires 2 of 3 keys"). However, in X.509/PKI, a certificate contains a single public key; multisig governance over how that key is used is an optional extra layer provided by HSMs or secret management systems. The policies in this extra layer, if they exist, are not knowable or verifiable by the public. This makes it impossible to distinguish between sloppy and robust management of secrets, and undermines fact-based reasoning about reputation and risk. Hackers thrive on opacity.

KERI takes a different tack. It allows identifiers governed by a single key, but if multiple keys will be used, it requires a definition of the identifier's fractionally weighted thresholds directly in the public KEL. This makes governance rules explicit and auditable. Each signing event, each rotation, and each change to signing policy occurs only as enacted by publicly verifiable cryptographic proof that the predeclared threshold policy was satisfied.

Scenario: the corporate board

Imagine a startup with three Founders (Alice, Bob, Carol) and four Investors (Dave, Eve, Frank, Genevieve). * Alice, Bob, and Carol each hold their own founder keys. * Dave, Eve, Frank, and Genevieve each hold their own investor keys.

The startup's publicly verifiable governance policy might state: "A rotation requires approval from a majority of Founders OR a supermajority of Investors."

In KERI, such a policy is defined in parallel arrays of an event data structure in the KEL. The controller lists all public keys in the `k` list, and then maps them to fractional weights in the `kt` field, which might look like this: `[["1/2", "1/2", "1/2"], ["1/3", "1/3", "1/3", "1/3"]]`

This sample array shows two subarrays or *clauses* (one for founders, one for investors) connected by OR logic. The weight of signers from at least one clause must sum to meet or exceed the threshold. Weights within each clause are summed, and the clause is satisfied if the sum of signing keys meets or exceeds 1.0. Thus, in the first subarray, any two founders provide enough weight to add up to 1.0; in the second subarray, any three investors can satisfy the threshold.

Scenario: break-glass recovery

A widely used pattern for individual users involves break-glass recovery. * Key A: Mobile Phone (Weight 0.5) * Key B: Laptop (Weight 0.5) * Key C: Cold Storage / Paper Wallet (Weight 1.0)

Threshold: 1.0

To sign a rotation, the user can combine Phone + Laptop ($0.5 + 0.5 = 1.0$). If the user loses their phone, they are not locked out. They can go to their safe, retrieve the cold storage key (Weight 1.0), and perform a rotation unilaterally. This allows for high-security recovery without relying on a "custodian" or a third party to reset a password. The recovery logic is baked into the math of the identifier itself [27].

Organizations already use multisig plus HSMs or key management systems to achieve security goals in nuanced, sophisticated ways. KERI supports that — but so does PKI. KERI's real innovation in multisig is less about *inventing* and more about *exposing*: multisig is optional and infinitely variable, but defining and following policy about it is required. This guarantees that the security of the whole ecosystem is explicit, auditable, and comparable.

2.5 Quantum resistance

Pre-rotation and governance agility provide a significant hedge against the quantum threat [28]. Quantum computers (using Shor's algorithm) threaten to break asymmetric encryption (like RSA and ECC) by deriving the private key from the public key.

However, quantum computers are not magic; they cannot easily invert a cryptographic hash function (like SHA-3 or Blake3). Grover's algorithm only provides a quadratic speedup for hashing, which is mitigated by using longer hashes (e.g., 256-bit) [28].

KERI takes advantage of these algorithm properties in two ways:

1. *Hiding the Target:* The rotation authority (the next key) is always hidden behind a hash. It is never exposed as a public key until the moment it is used and discarded. Therefore, even if a quantum attacker can crack the *active* key K1, they cannot crack the *next* key K2 because its public key is not yet visible to the network. By the time K2 is revealed in a rotation event, the controller has already moved trust to K3 (which is hashed). The root of control stays one step ahead of quantum analysis [26].
2. *Hybrid Governance:* Because KERI supports weighted multisig (see Section 2.4), a controller can employ a hybrid governance strategy. An identifier can be configured with a threshold that requires both a standard ECC signature (for efficiency and good tooling/interop today) and a post-quantum signature (for protections against surprise quantum advances). Even if the ECC key is broken, the attacker cannot satisfy the full threshold without also breaking the post-quantum key. Cryptographic agility lets KERI upgrade security postures without breaking the identifier's history [29].

2.6 Witnesses and the watcher network

Because KERI does not use a blockchain, it needs a mechanism to ensure availability and prevent *duplicity*. Duplicity is KERI's term for the double-spend problem in identity: a controller signing two different events with the same sequence number (e.g., rotating to Key A and also rotating to Key B at step 5).

Witnesses fill this role.

A witness is a server designated by the controller to store and serve the KEL. When a controller creates an event, they send it to their witnesses. The witnesses perform a sanity check (signature

and sequence valid?) and sign a receipt. Once a threshold of witnesses (e.g., 2 of 3) have signed, the event is stable.

Witnesses serve a fundamentally different role than CAs. A CA is trusted to attest to identity—to assert "this public key belongs to this entity." A witness makes no such assertion. A witness simply stores and serves events.

The trust model is adversarial: witnesses are assumed to be potentially malicious. To counter this, KERI uses *watchers*. Watchers are entities (run by verifiers or auditors) that periodically poll the witnesses. * Watcher asks Witness A: "What is the head of the log for Identifier X?" -> Witness A returns Event 5 (Hash X). * Watcher asks Witness B: "What is the head of the log for Identifier X?" -> Witness B returns Event 5 (Hash Y).

If Hash X and Hash Y differ, the watcher has detected duplicity. The watcher can broadcast the conflicting events as cryptographic proof of fraud. In the KERI ecosystem, this proof is fatal to the reputation of the identifier. Detection (rather than prevention) allows KERI to operate with low latency while ensuring that any dishonesty is provable [27].

3. ACDCs

While KERI handles the *identity* (the "Who"), *authentic chained data containers* (ACDCs) handle the *data* (the "What"). ACDCs are a protocol for attestations and verifiable credentials (VCs) that prioritize security, compactness, and provenance. They are designed to fix the copy-paste vulnerability of standard digital documents by turning data into a rigid, verifiable graph [30].

3.1 Issuance model

To understand how ACDCs are issued, let's step away from the abstract "Issuer/Holder" terminology and look at the engineering reality.

When a company issues an ACDC, its individual staff members do not manually sign a digital file. Rather, the process is managed by an automated agent running under their control. This agent must manage its cryptographic keys as KERI expects. 1. *Construction:* The API triggers the agent. The agent assembles the data payload (the attributes). 2. *Structuring:* The agent formats this data into the strict ACDC schema (more on this below). 3. *Anchoring:* The agent does not just sign the data. It creates a cryptographic digest (hash) of the ACDC and inserts it into the issuer's KEL (or a specialized TEL). This is an *anchored signature*. 4. *Transport* (*IPEX*)*:* The agent opens a secure channel to the holder's agent. It uses an issuance protocol like IPEX (issuance protocol for exchanging credentials). The issuer "offers" the credential. The holder "requests" it. The issuer "grants" it. 5. *Receipt:* The holder's agent receives the ACDC. The holder *also* anchors the receipt of the credential in their own KEL.

The issuer's log says "I issued Credential X at Sequence 50." The holder's log says "I accepted Credential X at Sequence 12." This mutual anchoring prevents spam (you can't force a credential into someone's wallet) and provides non-repudiation of receipt.

3.2 SAIDs

ACDCs rely on a unique cryptographic primitive called the *self-addressing identifier* (SAID).

In standard data handling, we often use UUIDs (random numbers) to identify records. The problem with a UUID is that it has no relationship to the data. If I change a field in the record, the UUID remains the same, but the data is different. Synchronization errors and version headaches follow.

A SAID is a content-addressable identifier. It is the cryptographic hash of the data structure itself. But there is a paradox: How can you include the hash of a file *inside* the file? If you write the hash into a field, you change the file, which changes the hash.

KERI solves this with a specific derivation algorithm. First, prepare the data structure as a template (JSON or CBOR map). In the ID field (`d`), insert dummy characters matching the target hash length (e.g., 44 # characters for Base64-encoded 256-bit hashes). Calculate the digest of this entire structure, then overwrite the dummy bytes with the calculated digest.

The identifier field matches the hash of the whole structure. If a single character of the payload changes, the SAID is invalidated, enforcing schema compliance and version control. An ACDC is not a mutable document; it is a crystallized fact [27].

3.3 Anchored vs. paired signatures

Many forms of signed digital evidence use a signing pattern that might be called *paired signatures*. A data structure format defines a payload plus its paired signature; the two go together because the format says so. For example, in the W3C Data Model, the signature lives in the `proof` attribute of the VC. [31] A JWT consists of headers, payload, and signature in a bundle. An X509 cert contains `tbsCertificate`, `signatureAlgorithm`, and `signature` (over `tbsCertificate`).

In contrast, ACDCs use *anchored signatures*. The signature may travel as a sidecar attachment (and thus resemble pairing), but independent of any container context, there's a provable association between the two because of an anchoring association in the KEL.

When a verifier checks an ACDC: 1. They extract the issuer's AID. 2. They retrieve the issuer's KEL from a witness. 3. They traverse the KEL to the sequence number specified in the signature. 4. They verify that the *digest* of the ACDC is present in that event.

The credential was issued *during* the active window of a specific key. If the key was rotated or compromised later, the anchor remains valid because it is ordered relative to key state changes by the sequence numbers in the log. This eliminates timestamp ambiguity. As NIST SP 800-102 states, a signed message "provides no assurance that the private key was used to sign the message at that time" [32], a limitation that traditionally necessitates complex external Time-Stamp Protocols [33]. Without the native anchoring provided by KERI, simple paired signatures are vulnerable to "retrograde attacks," where a compromised key is used to forge historical events [34].

3.4 Chained evidence: the graph of trust

The "C" in ACDC stands for "chained". In part, this refers to the ability to link credentials into a chain that traces derived authority to issue. X509s support this limited form of chaining. However, ACDC chaining allows any number of chains, with links expressing any type of connection with any desired weight. Thus, using ACDCs, you can build a directed acyclic graph (DAG) of arbitrarily rich semantics. The whole graph is verifiable and cacheable (modulo revocation tests)

as a unit or in any desired subset. ACDCs even offer a path to include foreign evidence types such as signed PDFs, scientific measurements, W3C VCs, X509 certificates, biometric readings, and so forth. [35]

Consider how this could upgrade trust in a supply chain:

1. *Mine in Australia:* Produces raw columbite-tantalite ore. Issues ACDC 1, a "Raw Ore" credential (schema = A) identified by its SAID (hash) = X. It documents the batch number and extraction date. ACDC 1 has two edges. One points to a credential proving the legal identity of the corporation operating the mine. The other points to a certification from RMI, stating that it has certified the mine as an ethical source under strong governance.
2. *Refiner in Thailand:* Buys the ore and refines it into tantalum. Issues ACDC 2, a "Refined Metal" credential (schema = B), SAID = Y. It documents the provenance of the ore that was smelted. ACDC 2 has two edges. One points to a credential proving the legal identity of the refiner. The other points back to ACDC 1 (by its SAID, X), asserting that the refined metal came from ethically sourced raw ore.
3. *Manufacturer:* Buys the tantalum and uses it to make a capacitor and in turn, a cell phone. Issues ACDC 3, a "Part" credential (schema = C), SAID = Z. ACDC 3 has one edge pointing back to ACDC 2 (by its SAID, Y).
4. *Retailer:* Sells the product. Issues ACDC 4, an "Ethically Sourced Product" ACDC, that points back to ACDC 3.

When a consumer scans the QR code on the final product, they receive the "Ethically Sourced Product" ACDC. Their verification software does a shallow verification of that credential, but it does not stop there. It follows all edges in the entire evidence graph, verifying signatures, anchors, and non-revocation status of ACDCs 3, 2, and 1, plus the unnumbered identity credentials and the certification from RMI. The consumer gains *transitive trust*; they know that the metal in the phone actually came from a conflict-free mine, not because the manufacturer said so, but because the cryptographic chain leads back to the mine's original issuance and its certification from an international governing body. Because SAIDs are immutable, no party in the middle can swap out a bad component for a good one without breaking the chain [30].

3.5 Privacy and selective disclosure

Despite this rigidity, ACDCs support privacy compliance (GDPR) through *selective disclosure*.

A well-designed ACDC payload is not a monolithic block. It is structured into sections (e.g., "Personal Info," "Medical Info," "License Info"). When the issuer creates the ACDC, they generate a random salt and a digest for each section. When the holder presents the credential to a verifier, they can use a Merkle-proof style strategy that reveal only the "License Info," with other sections represented only by their hash.

The KERI/ACDC architecture also resolves tensions with the right to be forgotten. In KERI, the KEL stores only the anchors (hashes) of the credentials. The actual data payload resides in the ACDC, which is held off-chain by the issuer and holder. When a user requests data deletion, the payload is destroyed. The hash remains in the log to preserve the integrity of the chain, but without the payload, the hash is irreversible. The identifier's history remains intact while the personal data is permanently removed [36].

This topological difference offers a distinct compliance advantage over a global database or a

blockchain-based identity. With a global database, conflicting regulatory constraints may make across-the-board compliance impossible. With a blockchain, data is replicated across all nodes; "forgetting" an identifier requires a hard fork of the entire chain. In KERI's micro-ledger architecture, an identifier is distinct from the network. A user can exercise the right to be forgotten by deleting their specific Key Event Log and revoking access at their witnesses, burning the identity without disrupting the global ecosystem.

4. CESR

The final pillar of the stack is Composable Event Streaming Representation (CESR). While KERI provides the logic and ACDC provides the container, CESR provides the language they speak. It is a serialization format designed specifically for the constraints of cryptographic transport.

4.1 The engineering bottleneck: text vs. binary

Engineers often face a choice between text formats (JSON, XML) and binary formats (Protobuf, ASN.1, CBOR). * *JSON:* Human-readable and easy to debug, but verbose and lacking native canonicalization. `{ "a": 1, "b": 2 }` and `{ "b": 2, "a": 1 }` are semantically identical but have different cryptographic hashes. While standards like JCS (RFC 8785) attempt to standardize this, the resulting normalization logic is brittle. In complex environments like JSON-LD, this fragility has led to "term redefinition" vulnerabilities, where the meaning of a signed credential can be altered without invalidating the signature [37]. * *Binary:* Compact and efficient, but opaque. You cannot look at a raw binary stream and know what it means without an external schema.

CESR bridges this gap using concatenation composability. It allows cryptographic primitives (keys, signatures, hashes) to be encoded in a text-safe way (Base64) that can be seamlessly converted to raw binary and back without breaking the signature.

4.2 Self-framing and pipelining

CESR is a cryptographic encoding format that achieves self-framing by incorporating type and size information directly into the data itself as a framing code prefix. JSON relies on delimiters (e.g., braces, commas) to parse structures.

A CESR parser operates efficiently by reading the initial characters (the code), consulting a set of defined code tables to determine the exact length of the cryptographic primitive or data object that follows. Self-framing allows a parser to read the payload and then immediately look for the next code.

CESR Primitives use short codes to conserve bandwidth. Here are examples of standard codes used within the KERI/ACDC protocol stack:

Code	Description	Data Size
A	Ed25519 256-bit seed (private key)	44
E	Blake3-256 cryptographic digest	44
0B	Ed25519 cryptographic signature	88
-A##	Generic pipeline group counter (small count)	4 (code length)

Making CESR primitives self-describing enables pipelining. Imagine a high-speed router processing millions of events. With JSON, the router must parse the entire object to find the "type" field. With CESR, the router reads the first few bytes. If it sees a prefix indicating a KEL event, it pipes the stream to the KEL processor. If it sees a prefix for an ACDC, it pipes it to the Credential processor. It acts like a cryptographic traffic cop, routing data streams without needing to fully deserialize or understand the payloads. For example, a stream beginning with `-A##` (a group counter) can be routed to batch processing logic, while a stream starting with `E` (Blake3-256 digest) can be routed to credential verification logic—all based on reading just the first 1-4 bytes. Edge environments (IoT, Telco) with expensive CPU and bandwidth require this capability [29].

4.3 Cryptographic agility

CESR's master code table provides *crypto agility*. Standard PKI struggles to upgrade algorithms (e.g., moving from RSA to ECC). Because the algorithm identifier is a signed attribute in X.509, an algorithm upgrade requires revoking and re-issuing entirely new certificates, a difficult process exemplified by the chaotic migration from SHA-1 to SHA-2 [38, 39].

In CESR, upgrading the ecosystem to post-quantum cryptography is as simple as associating new meaning to an unused prefix slot in an existing code table. Existing parsers will already know how many bytes this prefix consumes in the following stream, so they don't need to be re-released. A CESR implementation that predates the assignment of the `D` prefix to post-quantum Falcon-512 public keys will still handle the stream correctly: Read Code → Lookup Length → Read Bytes. Hybrid multisig can prevent surprise quantum attacks from assuming even temporary control of identifiers with some hackable key material. Witnesses and watchers guarantee that unauthorized usage is detected. Prerotation at a minimum guarantees that vulnerable identifiers have a recovery mechanism. The ecosystem has a straightforward, calm response to any quantum apocalypse [29].

5. Engineering implications and conclusion

Requirements discussed in identity circles, by regulators, and by cybersecurity experts—resilience, transparency, revocability, and privacy—are not achievable with the legacy X.509 stack. The administrative model of trust creates single points of failure that are difficult to insure and impossible to fully secure.

The KERI/ACDC/CESR stack offers a fundamental re-architecture: 1. *Resilience:* Pre-rotation and weighted multisig make the identity antifragile. It gets stronger with more keys, not more complex. 2. *Transparency:* The KEL provides a perfect, immutable audit trail. Duplicity is mathematically provable. 3. *Revocability:* ACDCs anchored in TELs allow for real-time status checks without privacy leaks. 4. *Efficiency:* CESR ensures this security model can run on constrained hardware with minimal overhead.

While verifying a KERI identifier theoretically has a cost relative to its history ($O(n)$), verification is highly efficient in practice due to incremental verification. Verifiers cache the state and only process new events since their last check ($O(\Delta n)$). For a decades-old identity that rotates quarterly, verifying history from any recent cache point is fast.

Adoption will likely proceed in parallel with the web PKI. KERI is positioned for domains where X.509's limitations are most acute: supply chain tracking, where provenance must survive across

organizational boundaries; IoT and edge computing, where devices require autonomous, long-lived identities; and high-assurance credentialing.

Engineers must shift their mental models. We stop thinking about certificates (static assertions) and start thinking about event streams (dynamic histories). We stop building admin panels for identity and start building agents that manage keys. Trust is not granted by a corporation or a government, but established by the mathematical consistency of the identifier itself.

Works cited

[1] Fox-IT. 2011. Operation Black Tulip: Report of the investigation into the DigiNotar Certificate Authority breach. Fox-IT, Delft, Netherlands.

[2] Sleevi, R. 2017. Chrome's Plan to Distrust Symantec Certificates. Google Security Blog. Retrieved December 18, 2024 from https://security.googleblog.com/2017/09/chromes-plan-to-distrust-symantec.html

[3] Resch, T. 2022. Mozilla Distrusts TrustCor Systems. Mozilla Security Blog. Retrieved December 18, 2024 from https://blog.mozilla.org/security/2022/11/30/mozilla-distrusts-trustcor-systems/

[4] Wang, W., Delavar, M., Azad, M. A., Nabizadeh, F., Smith, S., and Hao, F. 2024. Spoofing Against Spoofing: Towards Caller ID Verification In Heterogeneous Telecommunication Systems. ACM Trans. Priv. Secur. 27, 1, Article 5 (March 2024), 33 pages. https://doi.org/10.1145/3630107

[5] Office of Communications (Ofcom). 2024. Calling Line Identification (CLI) authentication: statement and further consultation. Ofcom, London, UK.

[6] Commission for Communications Regulation (ComReg). 2023. Combatting scam calls and texts: Consultation on network based interventions to reduce the harm from Nuisance Communications. Document 23/52. ComReg, Dublin, Ireland.

[7] Nohe, P. and Beattie, D. 2020. The CA Security Council Looks Ahead to 2020 and Beyond. PKI Consortium. Retrieved December 18, 2024 from https://pkic.org/tags/ca/browser-forum/

[8] Wilson, B. 2020. Ballot SC31 - Browser Alignment. CA/Browser Forum Public Mailing List. Retrieved December 18, 2024 from https://lists.cabforum.org/pipermail/servercert-wg/2020-June/002000.html

[9] Nohe, P. 2020. SSL Certificates: One Year Max Validity Ballot fails at the CA/B Forum. Hashed Out by The SSL Store. Retrieved December 18, 2024 from https://www.thesslstore.com/blog/ssl-certificates-one-year-max-validity-ballot-fails-at-the-ca-b-forum/

[10] Adams, C. and Lloyd, S. 2003. Understanding PKI: Concepts, Standards, and Deployment Considerations (2nd ed.). Addison-Wesley Professional, Boston, MA.

[11] Santesson, S., Myers, M., Ankney, R., Malpani, A., Galperin, S., and Adams, C. 2013. X.509 Internet Public Key Infrastructure Online Certificate Status Protocol - OCSP. RFC 6960. IETF.

[12] Liu, Y., Tome, W., Zhang, L., Choffnes, D., Levin, D., Maggs, B., Mislove, A., Schulman, A., and Wilson, C. 2015. An End-to-End Measurement of Certificate Revocation in the Web's PKI. In

Proc. ACM Internet Measurement Conference (IMC), 183-196. https://doi.org/10.1145/2815675.2815685

[13] Laurie, B., Langley, A., and Kasper, E. 2013. Certificate Transparency. RFC 6962. IETF.

[14] Scheitle, Q., Gasser, O., Nolte, T., Amann, J., Brent, L., Carle, G., Holz, R., Schmidt, T. C., and Wählisch, M. 2018. The Rise of Certificate Transparency and Its Implications on the Internet Ecosystem. In Proceedings of the Internet Measurement Conference (IMC '18). ACM, New York, NY, USA, 343–349. https://doi.org/10.1145/3278532.3278562

[15] Chrome Team. 2024. Chrome Certificate Transparency Policy. Google. Retrieved December 18, 2024 from https://googlechrome.github.io/CertificateTransparency/ct_policy.html

[16] Azevedo, L. C., Scheid, E. J., Franco, M. F., and Stiller, B. 2024. Assessing SSL/TLS Certificate Centralization: Implications for Digital Sovereignty. In Proceedings of the 2024 IEEE/IFIP Network Operations and Management Symposium (NOMS). IEEE, 1–6. https://doi.org/10.1109/NOMS59830.2024.10575394

[17] Sun, A., Lin, J., Wang, W., Liu, Z., Li, B., Wen, S., Wang, Q., and Li, F. 2024. Certificate Transparency Revisited: The Public Inspections on Third-party Monitors. In Proceedings of the Network and Distributed System Security Symposium (NDSS '24). The Internet Society, San Diego, CA. https://dx.doi.org/10.14722/ndss.2024.24834

[18] Barnes, R., Hoffman-Andrews, J., McCarney, D., and Kasten, J. 2019. Automatic Certificate Management Environment (ACME). RFC 8555. IETF.

[19] Aas, J., Barnes, R., Case, B., Durumeric, Z., Eckersley, P., Flores-López, A., Halderman, J. A., Hoffman-Andrews, J., Kasten, J., Rescorla, E., Schoen, S., and Warren, B. 2019. Let's Encrypt: An Automated Certificate Authority to Encrypt the Web. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19). ACM, New York, NY, USA, 2473–2487. https://doi.org/10.1145/3319535.3363191

[20] Hardman, D. 2025. The Evidence Lifecycle in Telco. Daniel Hardman's Papers. Retrieved December 18, 2024 from https://dhh1128.github.io/papers/ev-life.html

[21] Cooper, D., Santesson, S., Farrell, S., Boeyen, S., Housley, R., and Polk, W. 2008. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280. IETF.

[22] Olusegun, R. and Yang, B. 2024. Enhancing Blockchain Network Scalability Through Parallelization and Aggregation Techniques: A Survey. IEEE Access 12 (2024), 1-18. https://doi.org/10.1109/ACCESS.2024.3368256

[23] Mssassi, S. and El Kalam, A. A. 2025. The Blockchain Trilemma: A Formal Proof of the Inherent Trade-Offs Among Decentralization, Security, and Scalability. Applied Sciences 15, 1 (2025), 19. https://doi.org/10.3390/app15010019

[24] Smith, S. M. 2020. Key Event Receipt Infrastructure (KERI) Design and Build. Decentralized Identity Foundation.

[25] Smith, S. M. 2023. KID0001 - Prefixes, Derivation and derivation reference tables. Trust Over IP Foundation.

[26] Hardman, D. and Smith, S. M. 2025. KERI's Solution to Post-Quantum Security. Daniel Hardman's Papers. Retrieved December 18, 2024 from https://dhh1128.github.io/papers/kspqs.pdf

[27] Smith, S. M. 2024. KERI Specification (v2.7.0). Trust Over IP Foundation. Retrieved December 18, 2024 from https://trustoverip.github.io/kswg-keri-specification/

[28] Bernstein, D. J. 2009. Introduction to Post-Quantum Cryptography. In Post-Quantum Cryptography, D. J. Bernstein, J. Buchmann, and E. Dahmen, Eds. Springer, Berlin, 1-14.

[29] Smith, S. M. 2024. Composable Event Streaming Representation (CESR) Specification. Trust Over IP Foundation. Retrieved December 18, 2024 from https://trustoverip.github.io/kswg-cesr-specification/

[30] Smith, S. M. 2024. Authentic Chained Data Containers (ACDC) Specification. Trust Over IP Foundation. Retrieved December 18, 2024 from https://trustoverip.github.io/kswg-acdc-specification/

[31] Sporny, M., Longley, D., and Chadwick, D. 2022. Verifiable Credentials Data Model v1.1. World Wide Web Consortium (W3C).

[32] Barker, E. 2009. Recommendation for Digital Signature Timeliness. NIST Special Publication 800-102. National Institute of Standards and Technology, Gaithersburg, MD. https://doi.org/10.6028/NIST.SP.800-102

[33] Adams, C., Cain, P., Pinkas, D., and Zuccherato, R. 2001. Internet X.509 Public Key Infrastructure Time-Stamp Protocol (TSP). RFC 3161. IETF. https://doi.org/10.17487/RFC3161

[34] Hardman, D. 2025. Why Anchored Signatures? Daniel Hardman's Papers. Retrieved December 18, 2024 from https://dhh1128.github.io/papers/was.html

[35] Hardman, D. 2024. Bytewise and Externalized SAIDs. Daniel Hardman's Papers. Retrieved December 18, 2024 from https://dhh1128.github.io/papers/bes.pdf

[36] Hardman, D. 2025. ACDCs Should Underpin Digital Trust; Keep W3C VCs as Derivative Artifacts. Daniel Hardman's Papers. Retrieved December 18, 2024 from https://dhh1128.github.io/papers/acdc-vc-diff.html

[37] W3C Verifiable Credentials Working Group. 2024. Multiple significant security vulnerabilities in the design of data integrity. Issue #272. GitHub. Retrieved December 18, 2024 from https://github.com/w3c/vc-data-integrity/issues/272

[38] Google. 2014. Gradually sunsetting SHA-1. Google Security Blog. Retrieved December 18, 2024 from https://security.googleblog.com/2014/09/gradually-sunsetting-sha-1.html

[39] Cobb, M. 2017. All you need to know about the move from SHA-1 to SHA-2 encryption. CSO Online. IDG Communications. Retrieved December 18, 2024 from https://www.csoonline.com/article/550478/all-you-need-to-know-about-the-move-from-sha1-to-sha2-encryption.html